

Numerical Analysis Assignment 7

nijatabbasov06

November 2024

Table of Contents

1 Problem 8.3	2
2 Problem 8.4	6

1 Problem 8.3

The followings are the integration results for the integration of the 3 functions:

```
(a):  
Trapezoidal rule: 1.682884732142844  
Simpson's rule: 1.679246332776728  
Adaptive Simpson's rule: (1.6829421123469221, [])  
  
(b):  
Trapezoidal rule: 0.29422420136630023  
Simpson's rule: 0.29415752803230016  
Adaptive Simpson's rule: (0.2942257646203842, [])  
  
(c):  
Trapez rule: 1.333717012371756  
Simpson rule: 1.3269830056377487  
Adaptive rule: (1.333332871151053, [])
```

Figure 1: Integral Results

The followings are the images showing the evaluation points for the 3 functions.

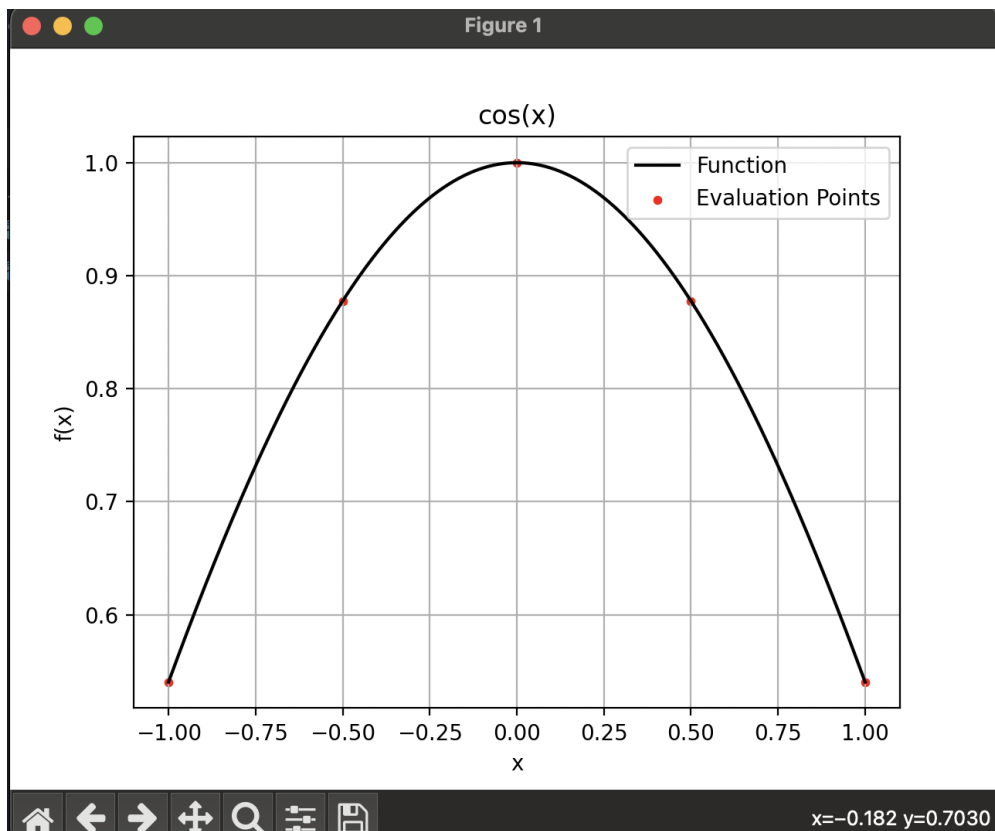


Figure 2: $\cos(x)$ plot for adaptive routine

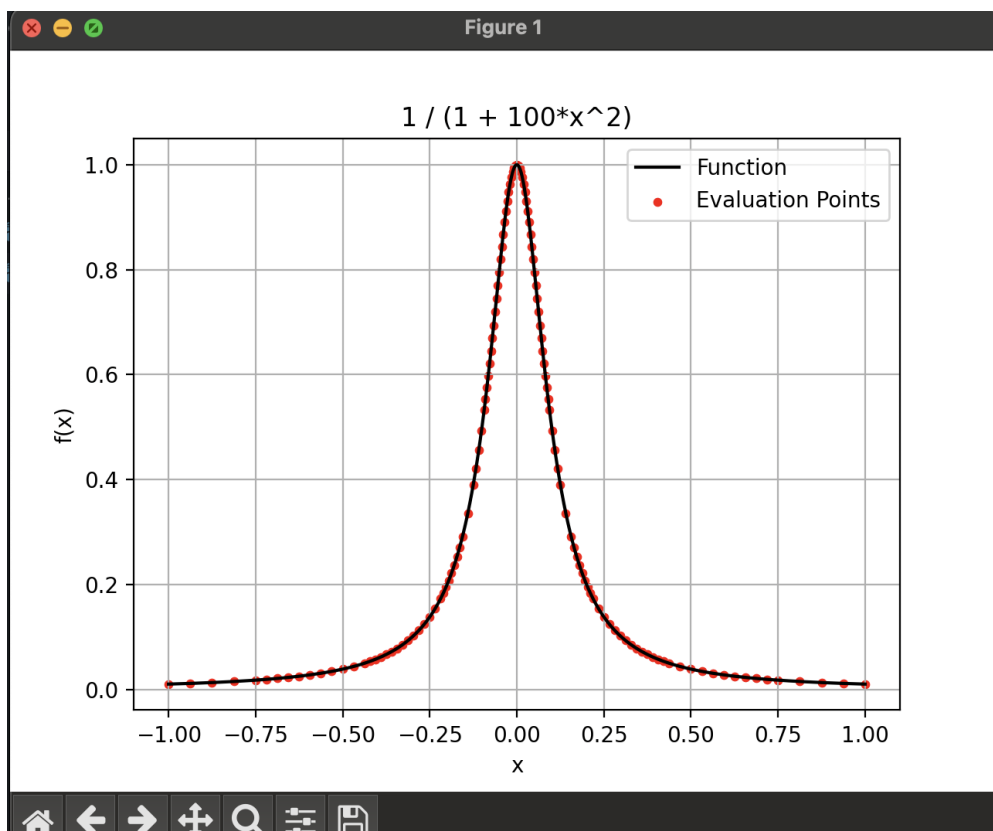


Figure 3: $\frac{1}{1+100x^2}$ plot for adaptive routine

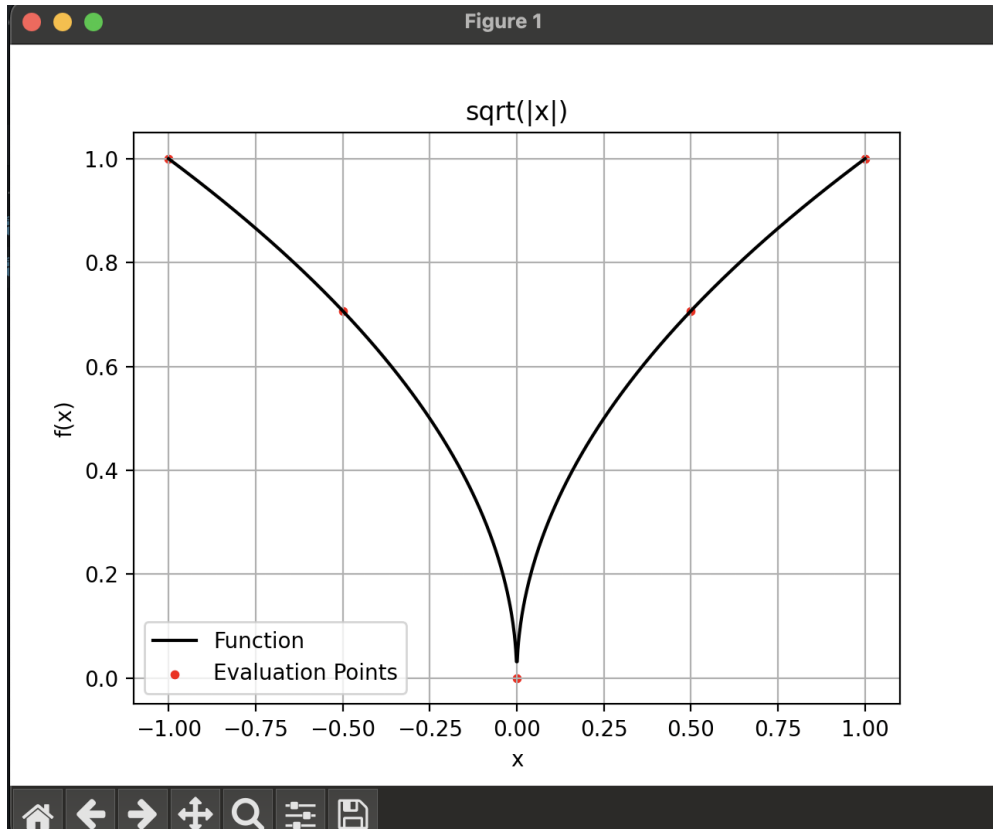


Figure 4: $\sqrt{|x|}$ plot for adaptive routine

In the following image, the error tolerations for each of the functions integrated is shown. While tol means error tolerations, title shows the function name. To elaborate, error tolerations of 10^{-4} , 10^{-7} , 10^{-2} are used for functions, $\cos(x)$, $\frac{1}{1+100x^2}$, $\sqrt{|x|}$, respectively.

```
tol=1e-4, title="cos(x)")

tol=1e-7, title="1 / (1 + 100*x^2)")

tol=1e-2, title="sqrt(|x|)")
```

Figure 5: Error Tolerations

2 Problem 8.4

The followings are the results of the integrations:

```
● (base) Nijats-MacBook-Pro:~ nijatabbasov$ /opt/anaconda3/bin/python "/Users/nijatabbasov/Desktop/KAIST_SEM_3/numerical analysis/assignment9/Pr_8.4.py"
Results (Simpson's Rule):
(a): 0.400000
(b): 0.399876
(c): 0.093930
(d): 0.499363
(e): 25.875023
(f): 1.000000
(g): -1.000376
○ (base) Nijats-MacBook-Pro:~ nijatabbasov$
```

Figure 6: Results of integrations

The followings are the results provided in the book:

8.4. Use numerical integration to verify or refute each of the following conjectures.

(a)

$$\int_0^1 \sqrt{x^3} dx = 0.4$$

(b)

$$\int_0^1 \frac{1}{1+10x^2} dx = 0.4$$

(c)

$$\int_0^1 \frac{e^{-9x^2} + e^{-1024(x-1/4)^2}}{\sqrt{\pi}} dx = 0.2$$

(d)

$$\int_0^{10} \frac{50}{\pi(2500x^2 + 1)} dx = 0.5$$

(e)

$$\int_{-9}^{100} \frac{1}{\sqrt{|x|}} dx = 26$$

(f)

$$\int_0^{10} 25e^{-25x} dx = 1$$

(g)

$$\int_0^1 \log(x) dx = -1$$

Figure 7: Results in the book

We can see that only the answer provided for the integral in c) is incorrect.

```

import numpy as np
import matplotlib.pyplot as plt
def f_a(x):
    return np.cos(x)
def f_b(x):
    return 1 / (1 + 100*x**2)
def f_c(x):
    return np.sqrt(np.abs(x))
def trap(f, a, b, n):
    x = np.linspace(a, b, n)
    y = f(x)
    h = (b - a) / (n - 1)
    integral = h * (y[0] + y[-1]) / 2
    for i in range(1, n-1):
        integral += h * y[i]
    return integral
def simp(f, a, b, n):
    x = np.linspace(a, b, n)
    y = f(x)
    h = (b - a) / (n - 1)
    integral = y[0] + y[-1]
    for i in range(1, n-1, 2):
        integral += 4 * y[i]
    for i in range(2, n-1, 2):
        integral += 2 * y[i]
    return h * integral / 3
def adapt(f, a, b, tol=1e-6, record_points=False):
    eval_points = []
    def simp_step(a, b, fa, fm, fb):
        h = b - a
        return h * (fa + 4*fm + fb) / 6
    def recursive(a, b, fa, fb, S, tol, level=0):
        m = (a + b) / 2

```

```

        integral += 4 * y[1]
    for i in range(2, n-1, 2):
        integral += 2 * y[i]
    return h * integral / 3
def adapt(f, a, b, tol=1e-6, record_points=False):
    eval_points = []
    def simp_step(a, b, fa, fm, fb):
        h = b - a
        return h * (fa + 4*fm + fb) / 6
    def recursive(a, b, fa, fb, S, tol, level=0):
        m = (a + b) / 2
        fm = f(m)
        f_left = f((a + m) / 2)
        f_right = f((m + b) / 2)
        S1 = simp_step(a, m, fa, f_left, fm)
        S2 = simp_step(m, b, fm, f_right, fb)
        S_total = S1 + S2
        if record_points:
            eval_points.extend([a, m, b])
        if abs(S - S_total) <= 15 * tol:
            return S_total
        else:
            return recursive(a, m, fa, fm, S1, tol/2, level+1) + \
                recursive(m, b, fm, fb, S2, tol/2, level+1)
    fa, fb = f(a), f(b)
    S = simp_step(a, b, fa, f((a + b) / 2), fb)
    result = recursive(a, b, fa, fb, S, tol)
    return result, sorted(set(eval_points))
def adapt_plot(f, a, b, tol, title):
    result, eval_points = adapt(f, a, b, tol, record_points=True)
    x = np.linspace(a, b, 1000)
    y = f(x)
    plt.plot(x, y, label='Function', color='black')
    y_points = [f(point) for point in eval_points]

```

```

result = recursive(a, b, Ta, Tb, S, tol)
return result, sorted(set(eval_points))
def adapt_plot(f, a, b, tol, title):
    result, eval_points = adapt(f, a, b, tol, record_points=True)
    x = np.linspace(a, b, 1000)
    y = f(x)
    plt.plot(x, y, label='Function', color='black')
    y_points = [f(point) for point in eval_points]
    plt.scatter(eval_points, y_points, color='red', s=10, label='Eval')
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()
a, b = -1, 1
print("(a):")
print(f"Trapezoidal rule: {trap(f_a, a, b, 100)}")
print(f"Simpson's rule: {simp(f_a, a, b, 100)}")
print(f"Adaptive Simpson's rule: {adapt(f_a, a, b)}")
print("\n(b):")
print(f"Trapezoidal rule: {trap(f_b, a, b, 100)}")
print(f"Simpson's rule: {simp(f_b, a, b, 100)}")
print(f"Adaptive Simpson's rule: {adapt(f_b, a, b)}")
print("\n(c):")
print(f"Trapez rule: {trap(f_c, a, b, 100)}")
print(f"Simpson rule: {simp(f_c, a, b, 100)}")
print(f"Adaptive rule: {adapt(f_c, a, b)}")
adapt_plot(f_a, a, b, tol=1e-4, title="cos(x)")
adapt_plot(f_b, a, b, tol=1e-7, title="1 / (1 + 100*x^2)")
adapt_plot(f_c, a, b, tol=1e-2, title="sqrt(|x|)")

```

```
def f_a(x):
    return np.sqrt(x**3)

def f_b(x):
    return 1 / (1 + 10 * x**2)

def f_c(x):
    return (np.exp(-9 * x) + np.exp(-1024 * (x - 0.25)**2)) / np.sqrt(np.pi)

def f_d(x):
    return 50 / (np.pi * (2500 * x**2 + 1))

def f_e(x):
    return 1 / np.sqrt(np.abs(x))

def f_f(x):
    return 25 * np.exp(-25 * x)

def f_g(x):
    return np.log(x)

print("Results (Simpson's Rule):")
print(f"(a): {simp(f_a, 0, 1):.6f}")
print(f"(b): {simp(f_b, 0, 1):.6f}")
print(f"(c): {simp(f_c, 0, 1):.6f}")
print(f"(d): {simp(f_d, 0, 10):.6f}")
print(f"(e): {simp(f_e, -9, 100):.6f}")
print(f"(f): {simp(f_f, 0, 10):.6f}")
print(f"(g): {simp(f_g, 1e-10, 1):.6f}")
```

```
import numpy as np
```

```
def simp(f, a, b, n=10000):  
    if n % 2 == 1:  
        n += 1  
    x = np.linspace(a, b, n + 1)  
    y = f(x)  
    h = (b - a) / n  
  
    integral = y[0] + y[-1]  
    integral += 4 * np.sum(y[1:n:2])  
    integral += 2 * np.sum(y[2:n-1:2])  
    return integral * h / 3
```

```
def f_a(x):  
    return np.sqrt(x**3)
```

```
def f_b(x):  
    return 1 / (1 + 10 * x**2)
```

```
def f_c(x):  
    return (np.exp(-9 * x) + np.exp(-1024 * (x - 0.25)**2)) / np.sqrt(np.pi)
```

```
def f_d(x):  
    return 50 / (np.pi * (2500 * x**2 + 1))
```

```
def f_e(x):  
    return 1 / np.sqrt(np.abs(x))
```

```
def f_f(x):  
    return 25 * np.exp(-25 * x)
```